



RAPPORT SPRINT 1

Générateur de grilles de Sudoku à difficulté contrôlée

Groupe X

KONCHEV Martin

DOLO Abdourahamane-Abdoul

BARRE-GUEROT Ilan

BRIOT-AMADIUE Kellyan

Encadrant : Marc Hartley

L3 informatique

Faculté des Sciences

Université de Montpellier

Décembre 2025

1. Compte rendu de la réunion avec l'encadrant

Date et heure de la réunion : 11 décembre à 14h00.

Participants : Tous les membres de l'équipe du projet Sudoku et notre encadrant.

Points clarifiés / décisions prises : Lors de cette première réunion, nous avons échangé sur les objectifs généraux du projet, à savoir la conception et la réalisation d'un jeu de Sudoku intégrant un système de gestion de la difficulté. Nous avons observé plusieurs exemples de serveurs Sudoku capables d'afficher toutes les possibilités pour une case donnée, ce qui nous a permis de mieux comprendre le fonctionnement interne d'un générateur et solveur de Sudoku.

L'encadrant nous a précisé les attentes pour le Sprint 1 et la structure du rapport à remettre. Nous avons également défini notre organisation pour les quatre prochains mois (jusqu'à la fin du mois d'avril) :

- fréquence des réunions (au moins une fois par semaine) ;
- contenu de ces réunions (bilan de l'avancement, planification des tâches, répartition du travail) ;
- suivi des progrès réalisés chaque semaine.

Cette rencontre nous a permis d'éclaircir tous les points essentiels du projet et de partager de nombreuses idées pour la suite du développement.

2. Cahier des charges

Description du sujet et des objectifs

Le projet consiste à créer un générateur de grilles de Sudoku capable de produire des grilles valides, avec une solution unique. Il vise également à estimer et contrôler la difficulté de ces grilles, en s'appuyant sur des méthodes de résolution automatiques (non nécessairement humaines pour ce sprint) via un solveur informatique. En complément, une interface de jeu en console, bien structurée qui permet de jouer directement sur les grilles générées et de tester les différentes fonctionnalités. L'objectif global est donc de disposer d'un outil qui génère des grilles, en évalue la difficulté et offre un environnement pratique pour les visualiser et les manipuler.

Périmètre retenu pour le projet

Le projet se concentre sur la génération de grilles de Sudoku complètes puis incomplètes, leur résolution et l'évaluation de leur difficulté. Nous incluons la mise en place d'un solveur, d'un générateur et de quelques techniques de résolution pour mesurer la complexité. En revanche, nous ne prévoyons pas de développer une interface utilisateur avancée ni un jeu complet pour le Sprint 1 : seule une interface minimale de test sera réalisée.

Fonctionnalités principales (MVP)

- **Génération de grilles valides :** produire automatiquement une grille complète puis une grille jouable avec solution unique.
- **Évaluation de la difficulté :** la difficulté dépend du nombre de backtracks observés pendant la résolution.
- **Parseur de fichiers :** lire et écrire des grilles de Sudoku à partir de fichiers texte.
- **Chiffres retirés :** permettre de retirer un certain nombre de chiffres qui dépend de la difficulté.
- **Solveur automatique :** permettre de vérifier qu'une grille est correcte, solvable et qu'elle n'a qu'une seule solution.
- **Interface minimale de test :** jouer un jeu de Sudoku dans le Terminal : afficher la grille, choisir la difficulté, insérer des valeurs, vérifier la validité.

Contraintes techniques et choix de technologies

- **Langage et outils :** le projet sera développé principalement en Python (pour la génération, le solveur, l'interface et les techniques de résolution).
- **Bibliothèques utilisées :** utilisation possible de modules standards (random, os) et d'outils de visualisation simples pour l'interface de test.

- **Contraintes de performance** : le générateur et le solveur doivent rester suffisamment rapides pour tester des grilles sans temps d'attente excessif.
- **Portabilité** : le code doit fonctionner sur les environnements Linux de l'université ainsi que sur les machines personnelles des membres du groupe.
- **Tests et validation** : mise en place de tests réguliers pour garantir que les grilles générées sont valides et qu'elles possèdent une solution unique ou plusieurs.
- **Organisation du dépôt Git** : adoption d'un workflow simple (branche principale + branches de développement), conventions de nommage et commits fréquents.

3. Première architecture du projet

Organisation générale

Plusieurs fichiers Python :

- contenant les fonctions principales du projet
- génération, manipulation, interface et résolution de grilles de Sudoku

Un répertoire sudoku.parser.out dédié aux données du projet :

Ce dossier contient plusieurs fichiers .txt utilisés comme supports d'entrée et de test :

- Generated.grille.completed.txt : une grille de Sudoku complète et valide
- Generated.grille.empty.txt : une grille de Sudoku vide
- Generated.grille.uncompleted.txt : des grilles partiellement remplies pour tester le solveur

Schéma simple ou description textuelle

Affichage de la structure globale du projet, avec les statistiques et la complexité. A la fin on obtient un grille de Sudoku valide et prêt pour être résolu avec une solution unique.

```
Grille vide :
. . . | . . . | . . .
. . . | . . . | . . .
. . . | . . . | . . .
-----
. . . | . . . | . . .
. . . | . . . | . . .
. . . | . . . | . . .
-----
. . . | . . . | . . .
. . . | . . . | . . .
. . . | . . . | . . .

Grille résolue :
4 1 9 | 8 6 3 | 7 5 2
2 3 8 | 5 1 7 | 6 9 4
6 7 5 | 4 2 9 | 3 1 8
-----
5 4 3 | 2 8 1 | 9 7 6
8 9 1 | 6 7 4 | 5 2 3
7 6 2 | 3 9 5 | 8 4 1
-----
1 8 6 | 9 5 2 | 4 3 7
9 2 4 | 7 3 6 | 1 8 5
3 5 7 | 1 4 8 | 2 6 9

Grille à résoudre:
. 1 9 | 8 6 . | . . 2
2 . . | 5 . 7 | . 9 4
. . . | . . . | 3 . .
-----
5 4 . | . . 1 | 9 7 6
8 . . | . 7 4 | . . .
. 6 2 | . 9 5 | 8 . 1
-----
. . . | . . 2 | 4 . .
. 2 4 | . . 6 | . 8 .
3 . 7 | . . 8 | . 6 .

--- Statistiques ---
Nombre de solutions trouvées : 1
La grille a une solution unique.
Difficulté de la grille : Facile
Nombre d'appels récursifs : 126
Nombre de tests effectués : 941
Nombre de backtracks : 80
La complexité est O(9^k) dans le pire des cas.
```

FIGURE 1 – Schéma de l'architecture pour le debut du projet

Interface Console :

```

=== SUDOKU CONSOLE ===
1. Générer une grille et jouer
2. Quitter
Votre choix: 1

Choisissez une difficulté:
1. Facile
2. Moyen
3. Difficile
4. Extrême
5. God Mode
Votre choix: 1

Grille prête à résoudre:
3 5 . | . . 1 | 4 . .
. 4 6 | . . 7 | 5 1 .
. . . | . 4 3 | . . 6
-----
6 . . | . 1 . | . 4 .
. 8 1 | 4 3 6 | 9 . .
. 9 4 | 8 2 5 | . 6 .
-----
. . 3 | 9 5 2 | 1 7 4
. 1 2 | . . 4 | 6 8 .
4 7 . | . . . | . . .

```

FIGURE 2 – Menu principal en console, choix de la difficulté et affichage d’une grille prête à être résolue

```

Actions disponibles:
1. Entrer un valeur
2. Résoudre automatiquement la grille
3. Quitter
Votre choix: 1
Ligne (1-9): 1
Colonne (1-9): 3
Valeur (1-9): 8

Valeur invalide pour cette position.

```

(a) Message d’erreur lors de la saisie d’une valeur invalide pour la case choisie

```

Actions disponibles:
1. Entrer un valeur
2. Résoudre automatiquement la grille
3. Quitter
Votre choix: 1
Ligne (1-9): 1
Colonne (1-9): 1
Valeur (1-9): 3

Cette case est déjà remplie.

```

(b) Message indiquant que la case sélectionnée est déjà remplie

```

Actions disponibles:
1. Entrer un valeur
2. Résoudre automatiquement la grille
3. Quitter
Votre choix: 1
Ligne (1-9): 1
Colonne (1-9): 3
Valeur (1-9): 7

Valeur insérée avec succès.
3 5 7 | . . 1 | 4 . .
. 4 6 | . . 7 | 5 1 .
. . . | . 4 3 | . . 6
-----
6 . . | . 1 . | . 4 .
. 8 1 | 4 3 6 | 9 . .
. 9 4 | 8 2 5 | . 6 .
-----
. . 3 | 9 5 2 | 1 7 4
. 1 2 | . . 4 | 6 8 .
4 7 . | . . . | . . .

```

(a) Insertion réussie d'une valeur dans la grille et mise à jour de l'affichage

```

Actions disponibles:
1. Entrer un valeur
2. Résoudre automatiquement la grille
3. Quitter
Votre choix: 2

La grille résolue automatiquement:
3 5 7 | 6 9 1 | 4 2 8
9 4 6 | 2 8 7 | 5 1 3
1 2 8 | 5 4 3 | 7 9 6
-----
6 3 5 | 7 1 9 | 8 4 2
2 8 1 | 4 3 6 | 9 5 7
7 9 4 | 8 2 5 | 3 6 1
-----
8 6 3 | 9 5 2 | 1 7 4
5 1 2 | 3 7 4 | 6 8 9
4 7 9 | 1 6 8 | 2 3 5

LE JEU EST TERMINÉ!

Veuillez appuyer sur une touche pour quitter.
Votre choix: █

```

(b) Résolution automatique complète de la grille

4. Organisation interne du groupe

Répartition des rôles

- **KONCHEV Martin** : Conception et implémentation des algorithmes de génération de grilles et du solveur de Sudoku (backtracking); Analyse des statistiques et de la complexité; Création de l'interface console; Rapport.
- **BRIOT-AMADIUE Kellyan** : Implémentation des algorithmes de parsing des grilles de Sudoku à partir de fichiers texte; Réalisation de tests fréquents afin de valider les fonctionnalités principales du projet.
- **BARRE-GUEROT Ilan** : Débogage du programme; Gestion du déroulement step by step dans l'interface console; Préparation de l'affichage des étapes successives et mise en place de la navigation dans l'historique des grilles.
- **DOLO Abdourahamane-Abdoul** : Mise en place d'une version fonctionnelle du parseur de grilles; Validation des données; Réalisation de tests unitaires pour vérifier des fonctions créées.

Outils utilisés

- Git (workflow, branches, revues) : <https://gitlab.etu.umontpellier.fr/e20230012387/ter-sudoku-groupX.git>
- Communication : Discord, réunions chaque semaine.

5. Planning prévisionnel

Découpage en étapes / jalons

- Sprint 1
- Fin Janvier
- Fin Février

- Fin Mars
- Fin Avril

Priorités des premiers sprints

Sprint 1 : Analyse du problème, recherche d’algorithmes, premières implémentations.

- compteur backtrack (nombre de calculs) pour avoir la solution : le nombre de backtrack peut donner une idée de la difficulté de la grille
- calculer la complexité théorique de l’algorithme de résolution
- générer une grille complète aléatoirement
- faire une première interface (debug, step by step) : préparer le futur, préparation pour pouvoir afficher la prochaine étape
- parseur de fichier : dossier avec fichiers spéciaux où on transforme des nombres en grille
- chaque grille générée doit avoir une solution unique

Tâches jusqu’au 30 avril (Gantt prévisionnel)

Fin Janvier :

Implémentation de techniques de résolution humaine

- Singleton (une seule possibilité)
- Singleton Caché

Fin Fevrier :

Implémentation de techniques de résolution humaine

- Paire cachée
- Triplet caché

Fin Mars :

Analyses :

- Comparaison difficulté des techniques avec complexité machine entre backtrack et résolution humaine

Fin Avril :

- Rédaction du rapport final
- Préparation de la soutenance (démonstration, présentation orale)

6. Prototype réalisé

Description de ce qui a été testé / mis en place (même minimal)

Pour ce premier sprint, un prototype fonctionnel a été mis en place, comprenant la génération de grilles valides, l’affichage interactif en console et l’intégration d’un solveur automatique. L’interface console permet déjà de jouer : insertion de valeurs, détection d’erreurs, gestion des cases remplies et affichage clair de la grille. Les tests réalisés ont confirmé que la génération produit une grille cohérente, que les contrôles de validité fonctionnent correctement et que le solveur est capable de compléter la grille, même si ses méthodes ne sont pas encore basées sur des stratégies humaines.

Justification des choix initiaux

- Les structures de données utilisées (matrices Python) et les algorithmes de résolution choisis (principalement backtracking) offrent une implémentation simple, fiable et suffisante pour valider rapidement les fonctionnalités essentielles.
- Les compromis ont privilégié la simplicité et la rapidité de développement : une interface console a été retenue pour faciliter les tests, et le solveur repose pour l’instant sur des méthodes automatiques plutôt que sur une modélisation complète des techniques humaines.